MỘC STUDIO / SEMINAR RABBITMQ

Hướng dẫn chạy, lời thuyết trình và giải thích code

Bản cửa hàng ngày 28/10/2026. Dùng cùng mã nguồn tại github.com/Sunn-dk/demo_soa_rabbitmq. Tên hàm là mốc tra cứu chính; số dòng có thể đổi ở phiên bản sau.

Chạy nhanh

```
git clone https://github.com/Sunn-dk/demo_soa_rabbitmq.git
cd demo_soa_rabbitmq
docker compose --profile kafka up -d --build --wait --wait-timeout 240
```

Cài Git và Docker Desktop, mở Docker trước. Web: http://localhost:8000. Hộp thư Mailpit: http://localhost:8025. RabbitMQ Management: http://localhost:15672 (guest / guest). Không cần cài Python/Node khi chạy Docker.

Bắt đầu với 0 worker. Đặt đơn, thêm consumer tạo hóa đơn, sau đó thêm consumer gửi email. Hai bước tách biệt để quan sát bất đồng bộ. Mailpit nhận SMTP và file PDF thật trong máy; không gửi thư ra Internet.

Các phần trong bản in

1. Kịch bản trình bày 15-20 phút và thao tác cụ thể.
2. Giải thích code theo luồng HTTP, outbox, PDF, email, ACK và phục hồi.
3. So sánh Kafka, giới hạn và nguồn chính thức.

Kịch bản Mộc Studio — 15–20 phút

Chạy docker compose --profile kafka up -d --build --wait --wait-timeout 240. Mở web cổng 8000 và Mailpit cổng 8025 trước giờ trình bày. Bắt đầu với 0 worker, chọn nhịp quan sát 4s. Nếu dùng dữ liệu cũ, dừng worker và reset cửa hàng trước. Không cần quay video.

0–3 phút: vấn đề thực tế và đặt hàng

Nói: “Một trang bán hàng cần ghi nhận đơn nhanh. Tạo PDF và gửi email là việc phía sau; nếu gộp hết vào HTTP request, khách phải chờ cả hai bước. Tôi sẽ đặt một đơn, rồi mở từng loại worker để thấy sự tách biệt.”

Thêm 2 áo thun, nhập thông tin ví dụ, đặt hàng. Tổng 498.000 + giao hàng 25.000 = 523.000 đ. Trang chuyển sang chi tiết đơn. HTTP 202 xuất hiện trong thông báo; có thể mở Network → checkout để chỉ status thật.

Nói: “202 nghĩa là server đã lưu đơn và yêu cầu xử lý vào cùng transaction. Chưa có worker nên chưa có hóa đơn. Dispatcher gửi yêu cầu trong outbox tới broker và chờ confirm; khác consumer ACK sau xử lý.”

3–6 phút: tách hai consumer

Sang Phòng vận hành → RabbitMQ. Chỉ queue hóa đơn có Ready. Thêm một consumer Tạo hóa đơn PDF, xem mã đơn trên thẻ worker và Unacked nếu kịp chu kỳ cập nhật. Sau đó chỉ ra PDF đã có, queue email còn chờ. Mở chi tiết đơn và xem hóa đơn.

Thêm consumer Gửi email kèm PDF, mở Mailpit, chọn thư theo số hóa đơn, tải file đính kèm.

Nói: “Đây là file PDF được tạo từ từng dòng hàng, không phải chỉ đổi trạng thái trên giao diện. Email đi qua SMTP và có file đính kèm. Mailpit giữ thư trên máy để cả lớp chạy không cần tài khoản email. Consumer hóa đơn và consumer email có thể tăng/giảm độc lập.”

6–9 phút: backlog và thêm worker

Đặt nhịp 4s, bấm Tạo 6 đơn hàng. Khi worker đầu đang bận, thêm consumer hóa đơn thứ hai rồi thứ ba. Chỉ các mã đơn khác nhau đang xử lý ở các process/PID khác nhau. Mở payload/outbox để thấy task_id, order_id, role.

Nói: “Queue đệm công việc khi đầu vào lớn hơn năng lực xử lý. prefetch=1 giới hạn mỗi consumer giữ một tác vụ chưa ACK. Chúng ta tăng năng lực tạo PDF mà không đổi API đặt hàng. Bốn giây ở đây là nhịp quan sát có thể tắt, không phải RabbitMQ truyền một tin mất bốn giây.”

9–12 phút: tin lỗi và phục hồi

Bấm Đơn lỗi email + 2 đơn tốt. Chỉ PDF của cả ba đã tạo, một tác vụ email sai ở DLQ, hai đơn tốt nhận thư. Sửa email và retry, mở thư vừa được gửi.

Bấm Sập sau lưu hóa đơn khi có từ 2 consumer hóa đơn; theo mã đơn trong log: INVOICE_COMMIT, CRASH_BEFORE_ACK, REDELIVER, DEDUP, ACK_SENT. Nếu chỉ có một consumer thì thêm consumer mới sau crash.

Nói: “ACK là consumer báo đã xử lý xong lượt giao. Consumer có thể đã lưu hóa đơn nhưng chết trước ACK nên broker giao lại. Mã đơn duy nhất và transaction giúp không tạo thêm hóa đơn. Đây không phải lời hứa exactly-once cho mọi thứ; SMTP và DB vẫn có một khoảng lỗi có thể làm gửi thư lặp.”

12–17 phút: Kafka cùng nghiệp vụ

1. Mở RabbitMQ & Kafka, bấm Chuẩn bị 4 worker hóa đơn mỗi bên và đợi thông báo sẵn sàng; bấm Nạp 6 đơn thật mỗi bên. Quan sát assignment: RabbitMQ có 4 consumer trên queue, Kafka có 4 process nhưng chỉ 2 process được giao partition. 2. Bấm Chạy tình huống lỗi trên cả hai broker. RabbitMQ gửi được hai thư tốt; Kafka giữ record email sai ở partition 0 nên hai record sau trong partition đó chưa gửi. 3. Vào phòng vận hành, chọn Kafka; bấm Chuyển sang topic lỗi. Chỉ KAFKA_APP_DLQ, OFFSET_COMMIT của record tiếp theo, hai email tốt xuất hiện. Có thể sửa và retry đơn đã chuyển lỗi. 4. Quay lại so sánh, bấm Đọc lại bằng group analytics mới. Chỉ group ID, partition, offset; tổng PDF/email không tăng.

Nói: “Với công việc gửi hóa đơn độc lập, RabbitMQ cho cơ chế ACK/NACK từng việc và DLX sẵn trong broker, thêm consumer trên queue thuận tiện. Kafka classic group gắn mức song song với partition. Ứng dụng này giữ thứ tự nên phải quyết định retry hoặc DLQ khi record lỗi. Kafka vẫn làm được; tăng partition hoặc xây retry/DLQ là phần thiết kế thêm. Khi cần lịch sử để ứng dụng khác đọc lại, Kafka lại thuận tiện.”

Không gọi đây là benchmark tốc độ. Hai bên dùng cùng code PDF/SMTP, số partition đã ghi rõ. Kafka Share Groups và RabbitMQ Streams là các mô hình khác ngoài phần đang chạy.

17–20 phút: chốt ý và để lớp thử

Chỉ README trên GitHub, lệnh clone và một lệnh Compose. Mỗi người chạy trên máy mình. Mời người nghe dự đoán: nếu tắt consumer email thì API đặt hàng, PDF và thư thay đổi thế nào? Sau đó thử ngay trên giao diện.

Nếu chỉ có 15 phút, rút phần chia việc còn 1 phút và giữ một tình huống crash; không bỏ bước mở email và file PDF vì đó là kết quả nghiệp vụ cụ thể.

Giải thích code theo từng phần demo Mộc Studio

Các vị trí dưới đây dùng tên hàm để tra cứu bền vững; số dòng là mốc của bản cửa hàng ngày 28/10/2026, có thể đổi sau khi sửa code. Lab v3 ở /lab có luồng khác. Mở file trong IDE, tìm tên hàm hoặc dùng rg -n 'def checkout' shop_api.py.

1. Bấm đặt hàng: HTTP 202 nằm ở đâu?

Trong static/shop.js, handler checkout-form.onsubmit gửi POST /api/shop/checkout bằng fetch và đọc response.status thật. shop_api.py dòng 120 khai báo @router.post('/checkout',status_code=202); hàm checkout() gọi store.create_order() rồi trả mã đơn, số tiền và ý nghĩa kết quả.

shop_store.create_order() dòng 125 kiểm tra khách, gọi quote() để lấy giá từ catalogue server, tính tổng và phí giao hàng. BEGIN IMMEDIATE dòng 138 bảo vệ thao tác kiểm tra/ghi khóa idempotency khỏi request đồng thời. Trong cùng transaction, code ghi orders và gọi enqueue() ghi tasks + outbox.

Vì sao viết vậy: nếu lưu đơn thành công nhưng publish broker thất bại, đơn không bị bỏ quên vì outbox vẫn còn để gửi lại. Nếu trình duyệt retry cùng idempotency key và cùng nội dung, server trả lại cùng đơn; dùng key cũ với nội dung khác bị từ chối. HTTP 202 chỉ nói đã tiếp nhận bền vào DB, chưa nói PDF đã tạo hoặc broker đã nhận.

2. Producer là ai, message chứa gì?

shop_api.dispatch() dòng 28 chạy một vòng lặp riêng cho mỗi transport, đọc pending_outbox() đúng broker, publish rồi store.published() khi có xác nhận. Hai vòng độc lập để Kafka hỏng không giữ việc RabbitMQ và ngược lại.

shop_store.enqueue() dòng 111 tạo message gọn: task_id, order_id, role. Dữ liệu khách/sản phẩm đã snapshot trong DB, worker đọc theo mã đơn. Thí nghiệm lỗi cùng partition thêm partition: 0, chỉ để chọn vị trí Kafka rõ ràng khi minh họa chính sách giữ thứ tự.

shop_broker.topology() khai báo direct exchange, queue durable, binding theo vai trò invoice/email, DLX và DLQ. rabbit_publish() đặt persistent message và mandatory routing; dispatcher bật confirm_delivery(). Confirm là broker xác nhận chặng publisher, độc lập với ACK của consumer.

Vì sao không nhét toàn bộ PDF vào message: broker phân phối yêu cầu làm việc; file được tạo/lưu ở nơi chuyên giữ dữ liệu. Demo chung một SQLite/file volume cho dễ chạy trên máy học; dịch vụ tách máy thật cần DB/API/object storage phù hợp.

3. Consumer hóa đơn làm gì? Hóa đơn lưu ở đâu?

Nút + Thêm consumer gọi /api/shop/workers; shop_api.add_worker() dòng 183 mở subprocess shop_worker.py <name> <transport> <role>. Đây là process riêng, không phải một thẻ HTML giả.

shop_worker.rabbit() đăng ký basic_consume(auto_ack=False) và basic_qos(prefetch_count=1). Callback nhận payload, ghi DELIVER/REDELIVER, rồi gọi shop_fulfillment.fulfill().

Với role invoice, fulfill() dòng 60 kiểm tra hóa đơn đã tồn tại chưa. Nếu có thì DEDUP và bỏ qua tạo thêm tác vụ email. Nếu chưa, render_invoice() dòng 29 dùng ReportLab tạo PDF: khách hàng, địa chỉ,

COD, mã đơn, số hóa đơn, từng dòng hàng, giá, tiền hàng, phí giao hàng và tổng. Font DejaVu Sans nhúng để hiển thị tiếng Việt. File tạm được os.replace() sang file cuối.

Lưu hai phần: bản ghi và snapshot hóa đơn ở bảng invoices trong data/shop.sqlite3; file ở data/invoices/<order_id>.pdf. API invoice_pdf() dòng 140 trả file bằng FileResponse. Modal xem hóa đơn lấy snapshot; nút Tải PDF tải file thật, không chụp HTML thành ảnh.

shop_store.commit_invoice() dòng 204 dùng order_id làm PRIMARY KEY, thêm bản ghi invoice và outbox email trong cùng transaction. Nếu transaction rollback thì cả hai cùng rollback. PDF trên đĩa không phải một phần transaction SQLite; file dư có thể được ghi lại khi retry.

4. ACK là gì và gửi tại dòng nào?

ACK là acknowledgement: consumer báo đã xử lý xong lượt giao. Trong shop_worker.py dòng 39, channel.basic_ack(method.delivery_tag) chạy sau fulfill(). delivery_tag xác định lượt giao trên channel, khác mã đơn nghiệp vụ. complete() ghi ACK_SENT sau khi gọi ACK thành công phía client.

Vì sao ACK sau xử lý: ACK sớm rồi worker chết có thể khiến broker loại bỏ việc chưa làm. ACK muộn cho phép giao lại khi worker chết, nhưng có khả năng xử lý lặp; vì thế cần idempotency. prefetch=1 là mỗi consumer chỉ giữ tối đa một delivery chưa ACK trong cấu hình này, không phải cả hệ thống chỉ xử lý một đơn.

5. Gửi email ở đâu? Có gửi Gmail không?

Tác vụ email được publish qua outbox sau commit hóa đơn. Role email cũng gọi fulfill() nhưng vào nhánh elif role=='email'. Nó kiểm tra sổ gửi store.sent(), xác thực email, đọc file PDF, tạo EmailMessage và add_attachment(...,maintype='application',subtype='pdf').

smtp.send_message(message) ở shop_fulfillment.py dòng 103 gửi SMTP thật tới service Mailpit trong Compose. Sau SMTP nhận thư, store.record_sent() ghi bảng deliveries và event SMTP_ACCEPTED. Worker mới ACK/commit sau đó. Mailpit UI cổng 8025 cho xem người nhận, nội dung và file đính kèm.

Vì sao dùng Mailpit: mọi người clone và chạy được mà không xin password email hay API key, không gửi ra người thật. Cơ chế SMTP vẫn có thật. Đây chưa phải kiểm tra email đến hộp thư Internet. SMTP và SQLite không có transaction chung; crash sau SMTP nhưng trước record_sent() có thể gửi trùng. Stable Message-ID hỗ trợ đối chiếu, không tự bảo đảm exactly-once.

6. Giả lập thời gian nằm ở đâu?

Setting observation_delay trong SQLite được đổi bằng /api/shop/settings. Ở shop_fulfillment.py dòng 74 và 92, pause(store.settings()['observation_delay']) giãn trước tạo PDF/gửi SMTP. Rabbit dùng conn.sleep để Pika phục vụ kết nối trong khi chờ; Kafka dùng time.sleep ngắn, nhỏ hơn max.poll.interval.

Chọn 0s để bỏ hoàn toàn giãn thời gian. PDF và SMTP vẫn chạy thật. Cả hai transport dùng cùng giá trị; đồng hồ trên trang có cả DB/dispatcher/worker/SMTP, không phải phép đo tốc độ riêng của broker.

7. REDELIVER và DEDUP xuất hiện thế nào?

Crash thủ công kill process khi task chưa ACK; RabbitMQ requeue delivery khi mất channel/connection. Consumer tiếp theo đọc cờ method.redelivered; shop_store.received() dòng 180 ghi REDELIVER và tăng lượt nhận.

Tình huống Sập sau lưu hóa đơn đặt crash_after_commit cho đơn. Trong fulfill(), sau commit invoice + email outbox, code ghi CRASH_BEFORE_ACK và gọi os._exit(42). Consumer khác nhận lại cùng task, thấy invoice đã có nên ghi DEDUP, không tạo thêm invoice/outbox email, rồi ACK.

Kafka chưa commit offset thì record cũng có thể được đọc lại sau rebalance; demo ghi REDELIVER dựa trên lịch sử task đã được nhận, không giả vờ Kafka có cùng cờ AMQP redelivered.

8. Email sai: DLQ của RabbitMQ và partition pause của Kafka

Seed ghi email sai để mô phỏng dữ liệu nhập qua kênh quản trị/tích hợp khác; checkout công khai vẫn chặn email sai. Handler email thật ném BusinessError ở bước validation. Trong shop_worker.rabbit(), lỗi xử lý dẫn tới basic_nack(requeue=False) dòng 35; cấu hình DLX route message vào queue .failed. Consumer tiếp tục nhận tin tốt.

shop_api.retry_task() lấy đúng message trong Rabbit DLQ, publish lại với confirm rồi ACK bản DLQ. Các message DLQ đã xem nhưng không phải đích được requeue khi đóng kết nối. Giữ cùng task ID để dedup; có thể publish lặp nếu crash ở khoảng giữa.

Trong shop_worker.kafka(), lỗi khiến consumer.pause() dòng 117 trên đúng partition, chưa commit record đó. Đây là chính sách giữ thứ tự của app. Nút sửa/retry xử lý lại message giữ trong worker. Nút chuyển topic lỗi publish error envelope, chờ delivery rồi consumer.commit() dòng 97 và resume partition. Hai đơn tốt phía sau mới đi tiếp; các partition khác vẫn có thể hoạt động.

9. Vì sao thêm Kafka worker chưa chắc tăng mức song song?

shop_broker.PARTITIONS=2. shop_worker.kafka() đăng ký on_assign/on_revoke, lưu partition thực của từng consumer. Với classic group và bốn consumer cùng topic hai partition, chỉ hai consumer có assignment. Có thể tăng partition hoặc thiết kế xử lý song song; demo đang cho thấy phần thiết kế thêm so với work queue có consumer nhận từng việc độc lập.

shop_api.analytics_replay() dòng 285 tạo group mới đọc log retained, trả order_id/partition/offset, không gọi nghiệp vụ tạo PDF/email. Đây là điểm mạnh Kafka; không loại khỏi bài chỉ để RabbitMQ trông tốt hơn. Đọc thêm SO_SANH_KAFKA.md.

10. Đối chiếu kết quả cụ thể

- HTTP 202: Network → POST checkout; response có order_id/total và nghĩa “đơn + outbox đã lưu”.

- Hóa đơn: bảng invoices + file PDF; unique order_id bảo vệ việc tạo bản ghi trùng.

- Email: bảng deliveries + Mailpit thư có attachment; bảng DB một dòng không đủ chứng minh SMTP không từng gửi lặp.

- Broker: Ready/Unacked/DLQ lấy Management API, có độ trễ; Kafka assignment lấy callback consumer thật.

- Luồng cùng mã đơn: ORDER_SAVED → PUBLISH_CONFIRMED → DELIVER → PDF_RENDER → INVOICE_COMMIT → ACK_SENT → tác vụ email → SMTP_ACCEPTED → ACK_SENT. Hai chặng có thể xen kẽ trong log vì process chạy đồng thời.

Nguồn chính thức: Work Queues, ACK/confirms, DLX, Kafka Design 4.2, Mailpit.

So sánh cho bài toán tạo hóa đơn và gửi email

Đối chiếu tài liệu chính thức ngày 28/10/2026. Demo cố định RabbitMQ 4.3.6 và Kafka 4.2.0; không khẳng định đây là bản mới nhất.

Điều kiện chung và khác biệt có chủ ý

Hai transport cùng catalogue, giá server, SQLite, shop_fulfillment.fulfill(), PDF ReportLab và SMTP Mailpit; nhịp quan sát cùng một setting. Mỗi hệ chạy một broker trên cùng máy. RabbitMQ dùng classic work queues, Kafka dùng Consumer Groups truyền thống, mỗi process xử lý tuần tự bản ghi đã poll. Kafka có 2 partition/topic, RF=1; không dùng Share Groups. Đây là so sánh mô hình phục vụ bài toán, không phải benchmark năng lực cực đại.

Bài toán: Thêm 4 worker xử lý các hóa đơn độc lập · RabbitMQ: 4 consumer cùng queue, prefetch=1 · Kafka: 4 consumer trong group, 2 partition → 2 consumer có assignment · Kết luận: RabbitMQ thêm mức song song mà không cần tăng partition cho bài toán này

Bài toán: Một email sai, hai email đúng phía sau · RabbitMQ: NACK riêng tin lỗi, DLX route tới DLQ; tin tốt tiếp tục · Kafka: Cùng partition 0, chính sách giữ thứ tự pause ở lỗi, chưa commit · Kết luận: RabbitMQ có ACK/NACK và dead-letter routing từng tin; Kafka app cần chiến lược offset/retry phù hợp

Bài toán: Khôi phục lỗi Kafka · RabbitMQ: Không áp dụng · Kafka: App publish .email.dead, đợi delivery, commit nguồn rồi resume · Kết luận: Kafka làm được DLQ; trong demo đây là code ứng dụng, không phải Kafka tự tạo DLQ

Bài toán: Sập sau lưu PDF trước xác nhận · RabbitMQ: Unacked được giao lại khi kết nối chết · Kafka: Record chưa commit được group đọc lại sau đổi owner · Kết luận: Cả hai cần idempotency với hiệu ứng bên ngoài broker

Bài toán: Thêm ứng dụng phân tích sau khi đã xử lý · RabbitMQ: Tin trong classic queue đã ACK không còn để consumer mới đọc lại · Kafka: Group analytics mới đọc lịch sử còn retention, không làm lại nghiệp vụ · Kết luận: Kafka log thuận tiện cho nhiều ứng dụng độc lập dùng lại lịch sử

Thí nghiệm 1: partition là đơn vị gán việc của classic group

shop_broker.PARTITIONS=2; mỗi vai trò có topic riêng. shop_worker.kafka() đăng ký callback on_assign, lưu assignment thực vào SQLite. UI đếm consumer có partition, không đếm process đang chạy thành năng lực xử lý. Nút chuẩn bị chờ 4 RabbitMQ consumer và 2 Kafka consumer có assignment trước khi báo sẵn sàng.

Có thể tăng partition để có 4 consumer hoạt động, hoặc lập lịch song song trong consumer; cần thiết kế xử lý offset khi hoàn tất lệch thứ tự và bảo vệ thứ tự theo key. Tăng partition cũng có ảnh hưởng phân bố key. Không diễn giải kết quả là Kafka chỉ chạy được hai worker hoặc RabbitMQ luôn nhanh hơn.

Thí nghiệm 2: lỗi nghiệp vụ khác lỗi broker

Seed tạo email thiếu dấu @ để handler thật từ chối; không ném lỗi riêng chỉ bên Kafka. Cùng ba đơn được route partition 0 trong thí nghiệm để dễ nhìn thấy thứ tự. Kafka callback giữ message lỗi trong bộ nhớ và pause đúng partition đó; vẫn poll giữ group membership. Partition khác không bị pause bởi quy tắc này.

Ứng dụng có hai cách khôi phục: sửa địa chỉ rồi xử lý lại record chưa commit, hoặc publish error envelope vào topic .dead, đợi xác nhận rồi commit nguồn và resume. Crash giữa publish DLQ và commit có thể sinh bản lỗi lặp. Kafka transactions có thể giải quyết một số luồng Kafka→Kafka; không tự làm SQLite/SMTP thành transaction Kafka.

RabbitMQ khai báo DLX và binding tới .failed; basic_nack(requeue=False) kích hoạt routing lỗi. Với classic queue một node, đây không phải bảo đảm tuyệt đối chặng DLQ không mất tin khi đích lỗi. Có thể chọn quorum queues và chính sách phù hợp khi triển khai thật.

Thí nghiệm 3: lợi thế log của Kafka

POST /api/shop/replay tạo group analytics ID mới, đọc topic hiện tại từ earliest còn retention và commit offset của group riêng. Chỉ đọc tối đa 100 record, không gọi fulfill(). Cửa hàng hiển thị bản ghi công việc (task), không giả vờ đó là hệ thống phân tích doanh thu đầy đủ. Tiền và hóa đơn trong DB không thay đổi vì đọc lại.

Muốn fanout RabbitMQ tới nhiều ứng dụng, có thể route bản sao tới các queue riêng ngay từ đầu. Muốn lưu log/replay theo mô hình RabbitMQ, xem Streams; không nói RabbitMQ nói chung không thể replay. Kafka Share Groups có chia sẻ record và xác nhận record, khác mô hình classic group của thí nghiệm.

Kết luận dùng để thuyết trình

“Trong luồng gửi hóa đơn gồm các công việc độc lập, RabbitMQ cho cách phân phối, xác nhận và cô lập lỗi trực tiếp ở mức message. Kafka mạnh khi coi dữ liệu là log để nhiều ứng dụng đọc và đọc lại. Sự phù hợp phụ thuộc mô hình xử lý, thứ tự, lịch sử và vận hành; không kết luận từ tên sản phẩm hoặc vài giây trên một laptop.”

Nguồn học chính thức

- RabbitMQ Work Queues: chia việc, manual ACK, persistence, prefetch.

- RabbitMQ ACK và publisher confirms: hai chặng xác nhận độc lập, Unacked và requeue khi kết nối mất.

- RabbitMQ Dead Letter Exchanges: điều kiện dead-letter, routing và giới hạn an toàn.

- Apache Kafka 4.2 Introduction: topic, partition, consumer groups và retention.

- Apache Kafka 4.2 Design: assignment theo partition, delivery semantics, offset và Share Groups.

- RabbitMQ Streams: mô hình log/replay khác classic work queue.

- Mailpit: SMTP, giao diện xem thư, attachments và API kiểm thử.